

Material de Aula

Curso: Bacharelado em Ciência da Computação

Disciplina: Tecnologia de Orientação a Objetos (TOO)

Professora: Vanessa Lago Machado

Programação Orientada à Objetos (POO)

11. Enum em Python – Conceito, Utilização e Exemplos Práticos

11.1. O que é Enum?

O termo **Enum** vem de *Enumeration* (ou *enumeração*).

Em Python, ele é definido através do módulo `enum` e permite criar **conjuntos de constantes nomeadas** — valores fixos que representam categorias ou estados possíveis.

O uso de **Enum** melhora:

-  a **legibilidade** do código,
 -  a **organização** de valores fixos,
 -  e reduz o risco de **erros por digitação** ou **valores inválidos**.
-

11.2. Como funciona

O módulo `enum` fornece uma classe base chamada `Enum` (do pacote `enum`), da qual podemos **herdar** para criar nossas enumerações personalizadas.

Cada elemento de um Enum é uma **constante imutável**, associada a um **nome** e, opcionalmente, a um **valor**.

Exemplo básico:

```
from enum import Enum

class StatusTarefa(Enum):
    A_FAZER = "A FAZER"
    EM_ANDAMENTO = "EM ANDAMENTO"
    CONCLUIDA = "CONCLUÍDA"
```

Usando:

```
print(StatusTarefa.A_FAZER)      # StatusTarefa.A_FAZER
print(StatusTarefa.A_FAZER.name)  # 'A_FAZER'
print(StatusTarefa.A_FAZER.value) # A FAZER
```

Cada item possui nativamente:

- `.name` → o **nome** do elemento da enumeração.
- `.value` → o **valor** associado.

11.3. Quando utilizar Enum

Use **Enum** sempre que precisar representar um **conjunto fixo de opções**, como:

- estados (ativo/inativo),
- níveis (baixo, médio, alto),
- tipos de tarefa (profissional, pessoal, gamer...),
- ou formas de pagamento, contrato, etc.

Isso torna o código mais **seguro e semântico**, substituindo valores mágicos como “ativo” ou 3 por constantes nomeadas.

11.4. Exemplo prático – Modelo de Tarefas

Suponha que temos uma classe Tarefa e queremos armazenar o **status** da tarefa (A FAZER, EM ANDAMENTO, CONCLUÍDA)

Podemos usar Enum para representar esses valores.

Exemplo:

```
from enum import Enum
from datetime import datetime

# --- Enumerações definidas ---
class StatusTarefa(Enum):
    A_FAZER = "A FAZER"
    EM_ANDAMENTO = "EM ANDAMENTO"
    CONCLUIDA = "CONCLUÍDA"
```

```
class Tarefa:
    def __init__(self, titulo, tipo: TipoTarefa, status=StatusTarefa.A_FAZER, data=None):
        self.titulo = titulo
        self.status = status
        self.data = datetime.strptime(data, "%d-%m-%Y") if data else None

    def concluir(self):
        self.status = StatusTarefa.CONCLUIDA

    def __str__(self):
        data_str = self.data.strftime("%d-%m-%Y") if self.data else "Sem data"
        return f"{self.titulo} [{self.status.value}] ({self.tipo.value}) - {data_str}"
```

Exemplo de uso:

```
t1 = Tarefa("Estudar Python")
t2 = Tarefa("Reunião do Projeto", StatusTarefa.EM_ANDAMENTO, "30-10-2025")
```

```
print(t1)
print(t2)

t1.concluir()
print("Após concluir:", t1.exibirDados)
```

🖨️ Saída esperada:

```
Estudar Python [A FAZER] (Pessoal) - Sem data
Reunião do Projeto [EM ANDAMENTO] (Profissional) - 30-10-2025
Após concluir: Estudar Python [CONCLUÍDA] (Pessoal) - Sem data
```

11.5. Boas práticas com Enum

- ✓ Use nomes em **maiúsculas** para os membros da Enum.
 - ✓ Prefira usar o `.name` e `.value` conforme o contexto (exibição, lógica).
 - ✓ Utilize Enum quando os valores possíveis **não devem ser alterados em tempo de execução**.
 - ✓ Evite misturar strings “solts” com enums — mantenha a coerência.
-